This shows that both Redis and our web service have stopped; the State column says Exit along with the status code the command terminated with (your exit status may be different). If, for some reason, either are still running, stop them with the docker-compose stop (or kill) command.

Now let's start up the entire app again—both the Rails server *and* Redis:

```
$ docker-compose up -d
```

Now if we run:

```
$ docker-compose ps
```

we can see both services are running:

```
Name                  Command                State         Ports
-------------------------------------------------------------------------------
myapp_redis_1  docker-entrypoint.sh redis…   Up      6379/tcp
myapp_web_1    bin/rails s -b 0.0.0.0        Up      0.0.0.0:3000->3000/tcp
```

Now, the moment of truth. Is our welcome#index action still connecting to the Redis server? Browse to http://localhost:3000/welcome again (or refresh the page if it's still open), and you should see the following familiar screen (but with the hit counter continuing to increase):

This page has been viewed 2 times!

Quick Recap

The true power of using containers for our apps is not running a process in an individual container (though that's useful), but rather how we're able to wire containers together so they can talk to each other.

In this chapter, we've seen how we can add services to our application, running in separate containers. More importantly, we've seen how Docker's built-in networking is used to let the services talk to each other.

Let's review the highlights:

1. We started a Redis server in a container, using docker run. We covered two new options: --name for giving containers a human-friendly name, and -d for running a container in detached mode.
2. We added a separate service in Compose for running the Redis server.
3. We verified that the Redis server was running (and that we could connect to it from a separate container) by starting a new container to run redis-cli.